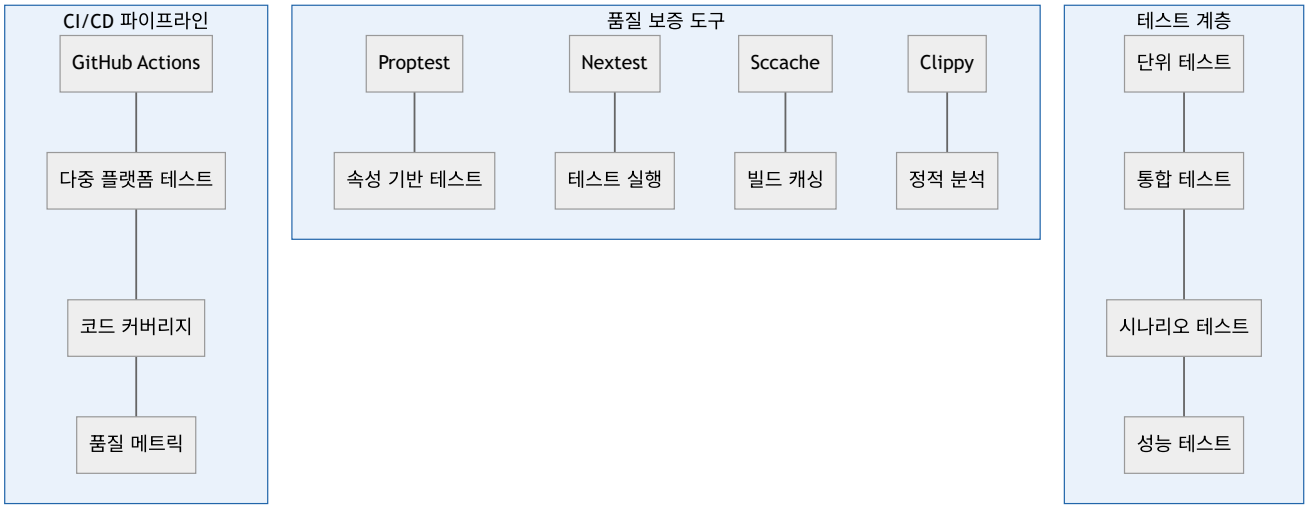
테스트 전략 및 품질 보증 분석

개요

iroh-blob 프로젝트는 분산 데이터 전송 시스템의 신뢰성과 성능을 보장하기 위해 다층적인 테스트 전략을 채택하고 있습니다. 이 문서는 단위 테스트, 통합 테스트, 시나리오 테스트, 속성 기반 테스트, 성능 테스트 및 품질 메트릭 측정 방식을 상세히 분석합니다.

테스트 아키텍처 개요



단위 테스트 구성과 역할

핵심 모듈별 단위 테스트

1. 프로토콜 레이어 테스트

- **위치**: `src/protocol/range_spec.rs`
- **목적**: 청크 범위 직렬화/역직렬화 검증
- **주요 테스트 케이스**:

```
#[test]
fn range_spec_wire_format() {
// 와이어 포맷 직렬화 검증
let cases = [
(RangeSpec::EMPTY, "00"),
(RangeSpec::all(), "01 00"),
// ... 다양한 범위 케이스
];
}
```

2. 저장소 구현체 테스트

- **위치**: `tests/blobs.rs`, `tests/tags.rs`
- **목적**: 메모리 저장소와 파일 시스템 저장소의 기본 기능 검증
- **테스트 패턴**:

```
async fn blobs_smoke(path: &Path, blobs: &Blobs) -> TestResult<()> {
// 바이트 추가/조회 테스트
// 파일 가져오기/내보내기 테스트
// 대용량 파일 진행 상황 테스트
```

```
}
```

3. 해시 및 검증 시스템 테스트

- **특징**: BLAKE3 해시 계산과 BAO 트리 검증
- **검증 항목**:
- 해시 일관성
- 청크 단위 검증
- 부분 데이터 무결성

흥미로운 크기 테스트 전략

프로젝트는 다양한 데이터 크기에 대한 체계적인 테스트를 위해 `INTERESTING_SIZES` 배열을 정의:

```
pub const INTERESTING_SIZES: [usize; 8] = [
0, // 빈 데이터 - 경계 조건
1, // 1바이트 - 최소 데이터
1024, // 정확히 1청크
1024 * 16 - 1, // 1청크 그룹 미만
1024 * 16, // 정확히 1청크 그룹
1024 * 16 + 1, // 1청크 그룹 초과
1024 * 1024, // 1MB - 중간 크기
1024 * 1024 * 8, // 8MB - 대용량
];
```

이 전략은 다음과 같은 경계 조건들을 체계적으로 테스트합니다:

- 청크 경계 (1024바이트)
- 청크 그룹 경계 (16KB)
- 인라인 저장 임계값
- 메모리 vs 파일 저장 전환점

통합 테스트 구성과 역할

클라이언트-서버 통신 테스트

1. 두 노드 간 블롭 전송 테스트

```
#[tokio::test]
async fn two_nodes_get_blobs_fs() -> TestResult<()> {
let (_testdir, (r1, store1, _), (r2, store2, _)) = two_node_test_setup_fs().awai
two_nodes_get_blobs(r1, &store1, r2, &store2).await
}
```

테스트 시나리오:

- QUIC 연결 설정
- 다양한 크기의 블롭 전송
- 데이터 무결성 검증
- 연결 종료 및 정리

2. 해시 시퀀스 전송 테스트

```
async fn two_nodes_hash_seq(
```

```
r1: Router, store1: &Store,
r2: Router, store2: &Store,
) -> TestResult<()> {
let  root = add_test_hash_seq(store1, sizes).await?;
store2.remote().fetch(conn, root).await?;
check_presence(store2, &sizes).await?;
}
```

검증 항목:

- 해시 시퀀스 생성 및 전송
- 종속 블롭들의 자동 다운로드
- 부분 다운로드 상태 관리

3. 다중 요청 처리 테스트

```
async  fn  two_nodes_get_many(
r1: Router, store1: &Store,
r2: Router, store2: &Store,
) -> TestResult<()> {
store2.remote().execute_get_many(
conn,
GetManyRequest::new(hashes, ChunkRangesSeq::all())
).await?;
```

```
}
```

저장소 구현체별 통합 테스트

각 저장소 구현체(메모리, 파일시스템)에 대해 동일한 테스트 스위트를 실행:

```
#[tokio::test]
async fn blobs_smoke_fs() -> TestResult {
let store = FsStore::load(td.path().join("a")).await?;
blobs_smoke(td.path(), store.blobs()).await?;
}

#[tokio::test]
async fn blobs_smoke_mem() -> TestResult {
let store = MemStore::new();
blobs_smoke(td.path(), store.blobs()).await?;
}
```

시나리오 테스트 구성과 역할

복잡한 실제 사용 시나리오

1. 부분 다운로드 및 복구 시나리오

```
async fn two_nodes_observe(
```

```
r1: Router, store1: &Store,
r2: Router, store2: &Store,
) -> TestResult<()> {
// 원격에서 부분 데이터 관찰
let mut stream = store2.remote().observe(conn.clone(), ObserveRequest::new(has
// 백그라운드에서 데이터 추가
store1.import_bao_bytes(hash, ChunkRanges::all(), bao).await?;
// 실시간 진행 상황 추적
while let Some(item) = stream.next().await {
current = current.combine(item?);
if current.is_validated() { break; }
}
}
```

2. 다중 소스 전환 시나리오

```
async fn three_nodes_blobs_downloader_switch() -> TestResult<()> {
// 노드 A: 부분 데이터만 보유
let ranges = ChunkRanges::from(..ChunkNum(17));
store1.import_bao_bytes(hash, ranges, bao).await?;
// 노드 B: 완전한 데이터 보유
let _tt2 = store2.add_bytes(data).await?;
// 노드 C: 두 소스로부터 다운로드
let request = DownloadRequest::new(
```

```
HashAndFormat::raw(hash),
[addr1.node_id, addr2.node_id]
);
}
```

3. 동시성 및 부하 테스트

- 다중 클라이언트 동시 접속
- 대용량 파일 전송 중 연결 중단
- 메모리 압박 상황에서의 동작

오류 처리 시나리오

1. 네트워크 오류 복구

- 연결 중단 시 재시도 로직
- 타임아웃 처리
- 부분 전송 실패 복구

2. 데이터 무결성 오류

- 해시 불일치 감지
- 손상된 데이터 처리
- BAO 검증 실패 대응

Proptest를 이용한 속성 기반 테스트 전략

Proptest 설정 및 구성

프로젝트는 `proptest = "1.6.0"` 을 사용하여 속성 기반 테스트를 구현합니다.

1. 청크 범위 속성 테스트

```
fn ranges(value_range: Range<u64>) -> impl Strategy<Value = ChunkRanges> {
prop::collection::vec((value_range.clone(), value_range), 0..16)
.prop_map(|v| {
let mut res = ChunkRanges::empty();
for (a, b) in v {
let start = a.min(b);
let end = a.max(b);
res |= ChunkRanges::chunks(start..end);
}
res
})
}

proptest! {
#[test]
fn range_spec_roundtrip(ranges in ranges(0..1000)) {
let spec = RangeSpec::new(&ranges);
let ranges2 = spec.to_chunk_ranges();
prop_assert_eq!(ranges, ranges2);
}
```

```
}
```

2. 직렬화/역직렬화 속성 테스트

```
proptest! {
#[test]
fn range_spec_seq_bytes_roundtrip(
ranges in proptest::collection::vec(ranges(0..100), 0..10)
) {
let expected = ranges.clone();
let actual = range_spec_seq_bytes_roundtrip_impl(&ranges);
prop_assert_eq!(expected, actual);
}
}
```

속성 기반 테스트의 장점

1. 경계 조건 자동 발견

- 수동으로 작성하기 어려운 엣지 케이스 자동 생성
- 다양한 입력 조합에 대한 체계적 테스트
- 회귀 테스트를 위한 실패 케이스 저장

2. 회귀 테스트 지원

```
proptest-regressions/protocol/range_spec.txt:
```

```
cc 5ff4de8531a81c637b4d202c97b724a41a989bc6894464e84db5ac2a519c08a9 # shrinks to

cc 50cb338763aa276705bb970d57d3d87e834f31a7e57bba810f46690c6d1e9955 # shrinks to
```

3. 비트필드 조합 테스트

```rust
#[proptest]
fn test_combine_order(
#[strategy(gen_non_empty_bitfields(32768, 4))] a: Bitfield,
#[strategy(gen_non_empty_bitfields(32768, 4))] b: Bitfield,
) {
// 비트필드 조합 연산의 교환법칙 검증
assert_eq!(a.clone().combine(b.clone()), b.combine(a));
}
```

엔티티 매니저 동시성 테스트

```rust
#[test_strategy::proptest]
fn test_counters_manager_proptest_mem(entries: Vec<(u64, u128)>) {
let rt = tokio::runtime::Builder::new_multi_thread().build().unwrap();
// 다양한 동시성 시나리오에서 카운터 매니저 테스트
}
```

성능 테스트와 벤치마크 방법론

성능 메트릭 수집

1. 내장 메트릭 시스템

```rust
#[derive(Debug, Default, MetricsGroup)]
#[metrics(name = "iroh-blobs")]
pub struct Metrics {
/// 다운로드된 총 바이트 수
pub download_bytes_total: Counter,
/// 다운로드에 소요된 총 시간 (ms)
pub download_time_total: Counter,
/// 성공한 다운로드 수
pub downloads_success: Counter,
/// 오류로 실패한 다운로드 수
pub downloads_error: Counter,
/// 찾을 수 없어 실패한 다운로드 수
pub downloads_notfound: Counter,
}
```

2. 다운로더 성능 메트릭

- `downloader_tick_main` : 메인 액터 루프 틱 수
- `downloader_tick_connection_ready` : 연결 준비 틱 수

- `downloader_tick_transfer_completed` : 전송 완료 틱 수
- `downloader_tick_transfer_failed` : 전송 실패 틱 수

성능 테스트 시나리오

1. 처리량 테스트

```
async fn test_export_ranges(
store: &Store,
hash: Hash,
data: &[u8],
range: Range<u64>,
) -> TestResult {
let start = Instant::now();
let actual = store
.export_ranges(hash, range.clone())
.concatenate()
.await?;
let duration = start.elapsed();
// 처리량 계산 및 기록
}
```

2. 메모리 사용량 테스트

- 대용량 파일 처리 시 메모리 사용량 모니터링
- 청크 단위 스트리밍의 메모리 효율성 검증

- 동시 연결 수에 따른 메모리 사용량 변화

3. 동시성 성능 테스트

```
async fn concurrent_download_test() {
let tasks: Vec<_> = (0..100).map(|_| {
tokio::spawn(async {
// 동시 다운로드 작업
})
}).collect();
let results = futures::future::join_all(tasks).await;
// 동시성 성능 분석
}
```

벤치마크 방법론

1. 마이크로 벤치마크

- 개별 함수의 성능 측정
- 해시 계산 속도
- 직렬화/역직렬화 성능
- 청크 범위 연산 성능

2. 시스템 벤치마크

- 전체 전송 파이프라인 성능
- 다양한 네트워크 조건에서의 성능

- 저장소 구현체별 성능 비교

3. 회귀 성능 테스트

- CI/CD 파이프라인에서 성능 회귀 감지
- 기준선 대비 성능 변화 추적
- 성능 임계값 기반 빌드 실패

코드 커버리지와 품질 메트릭

테스트 실행 환경

1. Nextest 설정

```
.config/nextest.toml:

[test-groups]
run-in-isolation = { max-threads = 32 }

[[profile.ci.overrides]]
filter = 'test(::run_in_isolation::)'
test-group = 'run-in-isolation'
threads-required = 32
```

특징:

- 격리된 테스트 실행으로 간섭 방지
- 리소스 집약적 테스트의 동시성 제어
- CI 환경에서의 안정적인 테스트 실행

2. 다중 플랫폼 테스트

GitHub Actions 매트릭스:

```
strategy:
matrix:
name: [ubuntu-latest, macOS-arm-latest]
rust: [stable]
features: [all, none, default]
```

품질 메트릭 측정

1. 정적 분석 도구

Clippy 설정:

```
- name: clippy check (all features)
run: cargo clippy --workspace --all-features --all-targets --bins --tests --benc
```

Deny 설정 (`deny.toml`):

```
[bans]
deny = [
"aws-lc", "aws-lc-rs", "aws-lc-sys",
"native-tls", "openssl",
]
```

```
[licenses]

allow = [

"Apache-2.0", "MIT", "BSD-3-Clause",

# ... 허용된 라이선스 목록

]
```

2. 코드 커버리지 측정

테스트 커버리지 전략:

- 단위 테스트: 개별 함수 및 모듈 커버리지
- 통합 테스트: 컴포넌트 간 상호작용 커버리지
- 시나리오 테스트: 엔드투엔드 워크플로우 커버리지

커버리지 도구:

- `cargo-tarpaulin` : Rust 코드 커버리지 측정
- `grcov` : LLVM 기반 커버리지 수집
- GitHub Actions 통합으로 자동화된 커버리지 리포팅

3. 품질 게이트

빌드 실패 조건:

- 컴파일 경고 (`RUSTFLAGS: -Dwarnings`)
- 문서 경고 (`RUSTDOCFLAGS: -Dwarnings`)
- Clippy 린트 실패
- 테스트 실패
- 라이선스 정책 위반

지속적 품질 개선

1. 자동화된 품질 검사

```yaml
env:
RUST_BACKTRACE: 1
RUSTFLAGS: -Dwarnings
RUSTDOCFLAGS: -Dwarnings
SCCACHE_CACHE_SIZE: "50G"
```

2. 성능 회귀 방지

- 벤치마크 결과의 기준선 추적
- 성능 임계값 기반 알림
- 성능 영향 분석 자동화

3. 의존성 관리

- `cargo-deny` 를 통한 의존성 정책 강제
- 보안 취약점 자동 스캔
- 라이선스 호환성 검증

테스트 전략의 효과성 분석

버그 발견 및 예방

1. 속성 기반 테스트의 성과

- 수동 테스트로 발견하기 어려운 엣지 케이스 자동 발견

- 회귀 테스트 케이스 자동 생성 및 보존
- 코드 변경 시 예상치 못한 부작용 조기 발견

2. 통합 테스트의 가치

- 실제 네트워크 환경에서의 동작 검증
- 다중 컴포넌트 간 상호작용 문제 발견
- 성능 병목점 및 리소스 누수 감지

3. 시나리오 테스트의 중요성

- 실제 사용자 워크플로우 검증
- 복잡한 오류 상황에서의 복구 능력 확인
- 시스템 한계 상황에서의 안정성 검증

개발 생산성 향상

1. 빠른 피드백 루프

- Nextest를 통한 병렬 테스트 실행
- Sccache를 통한 빌드 시간 단축
- 격리된 테스트 실행으로 안정성 향상

2. 자동화된 품질 보증

- CI/CD 파이프라인에서의 자동 품질 검사
- 다중 플랫폼 호환성 자동 검증
- 의존성 및 라이선스 정책 자동 강제

3. 유지보수성 개선

- 포괄적인 테스트 커버리지로 리팩토링 안전성 확보

- 명확한 테스트 구조로 코드 이해도 향상
- 회귀 테스트로 기존 기능 보호

결론

iroh-blob 프로젝트의 테스트 전략은 다층적이고 포괄적인 접근 방식을 통해 높은 품질과 신뢰성을 달성하고 있습니다. 단위 테스트부터 복잡한 시나리오 테스트까지, 그리고 속성 기반 테스트를 통한 자동화된 엣지 케이스 발견까지, 각 테스트 레벨이 고유한 가치를 제공하며 전체적인 품질 보증 체계를 구성합니다.

특히 proptest를 활용한 속성 기반 테스트는 전통적인 예제 기반 테스트로는 발견하기 어려운 버그들을 효과적으로 찾아내며, 회귀 테스트 케이스를 자동으로 생성하여 장기적인 코드 품질 유지에 기여하고 있습니다.

성능 테스트와 메트릭 수집 시스템은 시스템의 성능 특성을 지속적으로 모니터링하고 회귀를 방지하는 데 중요한 역할을 하며, 자동화된 CI/CD 파이프라인은 모든 품질 검사를 일관되게 수행하여 개발팀의 생산성을 높이고 있습니다.

이러한 종합적인 테스트 전략은 iroh-blob이 분산 환경에서 안정적이고 효율적으로 동작할 수 있는 기반을 제공하며, 지속적인 개발과 유지보수를 위한 견고한 품질 보증 체계를 구축하고 있습니다.